UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA
DE SISTEMAS Y COMPUTACIÓN

SEMANA 01: MANUAL GITHUB

TALLER DE DESARROLLO

DE APLICACIONES I

ADAUTO TORPOCO Cristian Pedro

HUANCAYO

2026

1) Fundamentos: El Motor y la Nube

El primer paso para dominar el control de versiones es entender que **Git y GitHub no son lo mismo**, aunque trabajen en conjunto.

- **Git (El Motor):** Es un sistema de control de versiones distribuido que se instala en tu computadora de forma local. Su trabajo es monitorear los archivos de tu proyecto, registrar cada cambio que realizas a lo largo del tiempo y permitirte viajar al pasado si cometes un error o rompes el código. Todo esto ocurre sin necesidad de internet.
- **GitHub (La Nube):** Es una plataforma de alojamiento web diseñada específicamente para repositorios de Git. Funciona como un "Google Drive" pero optimizado para código. Permite tener una copia de seguridad remota de tu trabajo, facilitar el trabajo en equipo, revisar código visualmente y, como en el caso de tu portafolio, alojar páginas web estáticas mediante *GitHub Pages*.

2) Configuración Inicial

Antes de comenzar a versionar un proyecto, Git necesita saber quién eres. Esto es vital porque en un entorno profesional o académico, cada línea de código modificada debe tener la "firma" de su autor para fines de auditoría y trazabilidad.

- Abre tu terminal (o la consola de IntelliJ/VS Code) y ejecuta los siguientes comandos por única vez en tu computadora:

git config --global user.name "Tu Nombre Completo"
git config --global user.email tu_correo@universidad.edu.pe

3) Flujo de Trabajo Diario: Los Comandos Esenciales

El ciclo de vida de un archivo en Git pasa por tres estados principales: modificado (en tu disco duro), preparado (en el *Staging Area*) y confirmado (guardado permanentemente en la base de datos de Git). Para navegar por este flujo, utilizaremos los siguientes comandos:

- `git init`: **El inicio de todo.** Se ejecuta en la raíz de la carpeta de tu proyecto. Crea un directorio oculto llamado `.git` que convierte una carpeta normal en un repositorio capaz de rastrear cambios.
- `git status`: **El radar.** Es el comando más utilizado. Te muestra qué archivos han sido creados, cuáles han sido modificados (en color rojo) y cuáles están listos para ser guardados (en color verde).
- `git add`: **Preparando el paquete.** Mueve los archivos modificados al *Staging Area* (área de preparación).

- `git commit`: **La fotografía permanente.** Toma todo lo que está en el área de preparación y lo guarda en el historial del repositorio con un código único (hash). Siempre debe ir acompañado de un mensaje claro:
- `git log`: **La máquina del tiempo.** Muestra una lista detallada de todos los commits realizados, indicando quién hizo el cambio, en qué fecha y cuál fue el mensaje. Es útil para auditar el avance del proyecto semana a semana.

4) Gestión de Ramas (Branches)

En el desarrollo de software, la regla de oro es: **"Nunca trabajes directamente en la rama principal (main o master)"**. Las ramas permiten crear copias de tu código para trabajar en nuevas funcionalidades de forma aislada.
Imagina que tu aplicación ya funciona y quieres añadir una nueva ventana gráfica en Java Swing. Si lo haces en la rama principal y algo sale mal, tu aplicación dejará de funcionar.

- `git branch <nombre>`: Crea una nueva rama paralela. Ejemplo: `git branch vista-swing`.

- `git switch <nombre>` (o `git checkout`): Te permite "saltar" de la rama actual a la nueva rama creada para empezar a trabajar de forma segura.

- `git merge <nombre>`: Una vez que tu nueva funcionalidad está terminada y probada en su propia rama, vuelves a tu rama principal (`git switch main`) y usas este comando para fusionar e integrar el código nuevo.

5) Trabajo Remoto: Conectando Local con GitHub

Una vez que tienes el control local de tu código, el siguiente paso es sincronizarlo con la nube (GitHub). Esto te protege contra pérdidas de datos y te permite acceder a tu código desde cualquier equipo.

- **Clonar un repositorio (`git clone`):** Si el proyecto ya existe en GitHub, puedes descargar una copia idéntica a tu computadora usando la URL del repositorio: `git clone` https://github.com/tu-usuario/mi-portafolio.git

- **Vincular un repositorio existente (`git remote add`):** Si creaste tu proyecto primero en tu computadora y luego hiciste el repositorio en GitHub, debes enlazarlos: `git remote add`

`origin` https://github.com/tu-usuario/mi-proyecto.git

- **Subir código (`git push`):** Envía tus *commits* locales al repositorio en la nube. `git push origin main` (Sube los cambios a la rama principal).
- **Descargar código (`git pull`):** Si trabajas desde otra computadora o un compañero hizo cambios en el repositorio, este comando descarga las novedades y las fusiona con tu código local para mantenerte actualizado.

6) Flujo Colaborativo (Forks y Pull Requests)

Cuando trabajas en equipos grandes o participas en proyectos *Open Source* (Código Abierto), no se te da acceso directo para modificar el código principal. En su lugar, se utiliza el flujo profesional estándar:

- **Fork:** Haces una copia exacta del repositorio de la empresa o proyecto en tu propia cuenta de GitHub.
- **Clone y Trabajo:** Clonar tu *Fork* localmente, creas una rama y programas la solución o mejora.
- **Push:** Subes esos cambios a TU repositorio en GitHub.
- **Pull Request (PR):** Desde GitHub, envías una solicitud formal al dueño del proyecto original diciendo: *"He mejorado este código, por favor revísalo e intégralo"*. El equipo revisará tu código, hará comentarios y, si todo es correcto, aprobarán tu PR, fusionando tu trabajo con el proyecto principal.

7) Buenas Prácticas: El uso vital del archivo .gitignore

Uno de los errores más comunes y destructivos en el desarrollo es subir archivos basura, compilados o contraseñas al repositorio público. GitHub solo debe almacenar **código fuente**.
Para evitar subir archivos no deseados, se crea un archivo de texto plano llamado `.gitignore` en la raíz del proyecto. Cualquier ruta escrita en este archivo será completamente ignorada por Git.

Ignorar archivos compilados de Java (No subir los .class)
*.class

Ignorar la carpeta autogenerada por Maven (Archivos pesados e innecesarios)
Target/

Ignorar configuraciones específicas de tu entorno de desarrollo (IDE)
Si tu profesor usa Eclipse y tú IntelliJ, subir esto causará conflictos
.idea/ *.iml .vscode/ .classpath .project

Ignorar archivos del sistema operativo
.DS_Store
Thumbs.db